

DRAFT

Prerequisites:

Minikube dari modul sebelumnya

☰ Modul Praktikum Containerization & Pengantar Cloud

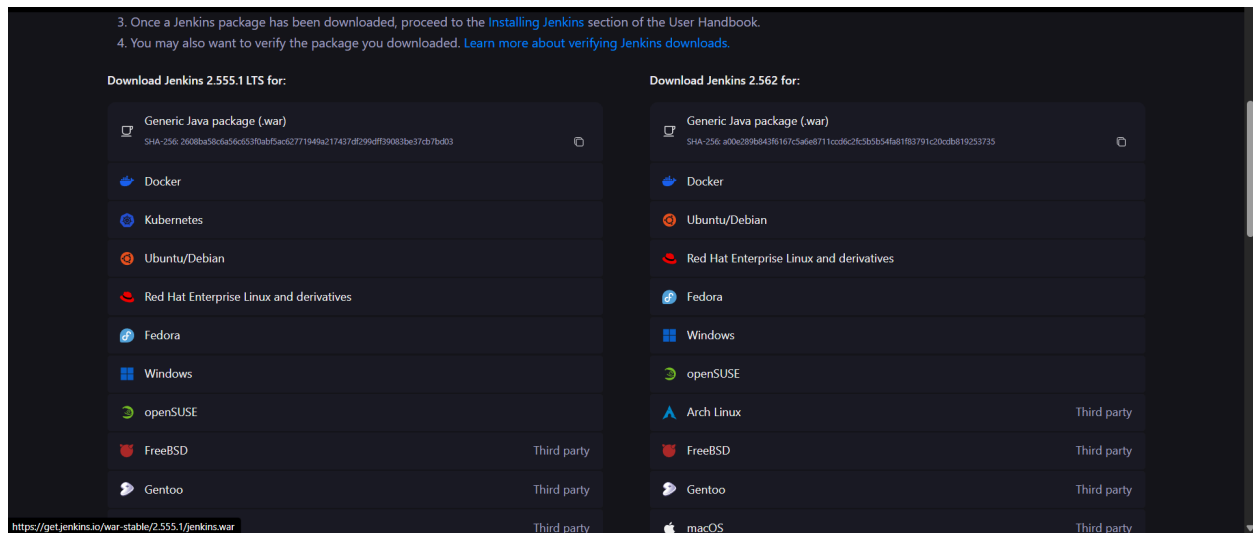
[Download Jenkins](#) (Jenkins 2.555.1 LTS)

[Java Downloads | Oracle](#) (OpenJDK 21), cek java -version, pastikan 21.x.x

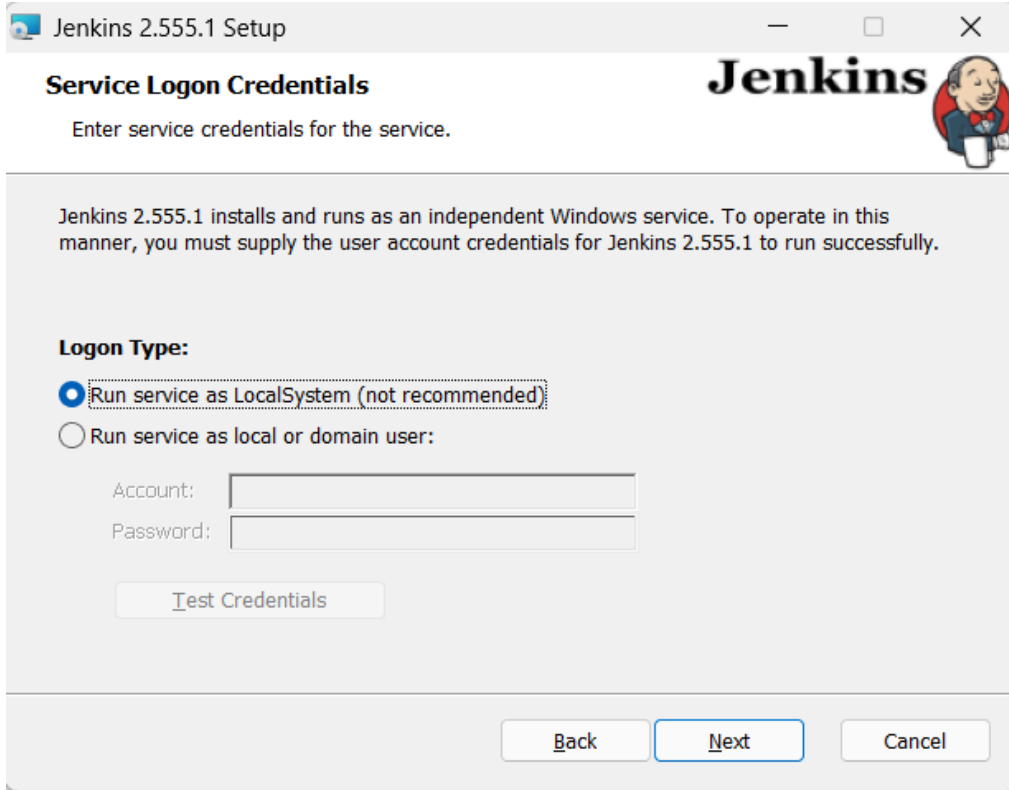
PREPARATION

Jenkins

Pilih yang 2.555.1 LTS yang Windows (kalo pake windows)



Kalau sudah, kita masuk tahap instalasi. Next saja sampai Service Logon Credentials lalu pilih Run Service as LocalSystem (untuk kepentingan praktikum). Ini memberikan Jenkins "master key" agar dia punya izin menjalankan Docker dan Kubectl di device tanpa bolak-balik minta izin admin.



The image shows a Windows installer window titled "Jenkins 2.555.1 Setup". The window has a standard Windows title bar with minimize, maximize, and close buttons. The main content area is titled "Service Logon Credentials" and includes the Jenkins logo (a cartoon character with a red bow tie) in the top right corner. Below the title, there is a instruction: "Enter service credentials for the service." A paragraph of text states: "Jenkins 2.555.1 installs and runs as an independent Windows service. To operate in this manner, you must supply the user account credentials for Jenkins 2.555.1 to run successfully." Under the heading "Logon Type:", there are two radio button options. The first option, "Run service as LocalSystem (not recommended)", is selected with a blue dot. The second option is "Run service as local or domain user:". Below these options are two text input fields labeled "Account:" and "Password:". A "Test Credentials" button is located below the password field. At the bottom of the window, there are three buttons: "Back", "Next" (which is highlighted with a blue border), and "Cancel".

Jenkins 2.555.1 Setup

Service Logon Credentials

Enter service credentials for the service.

Jenkins 2.555.1 installs and runs as an independent Windows service. To operate in this manner, you must supply the user account credentials for Jenkins 2.555.1 to run successfully.

Logon Type:

☒ Run service as LocalSystem (not recommended)

☐ Run service as local or domain user:

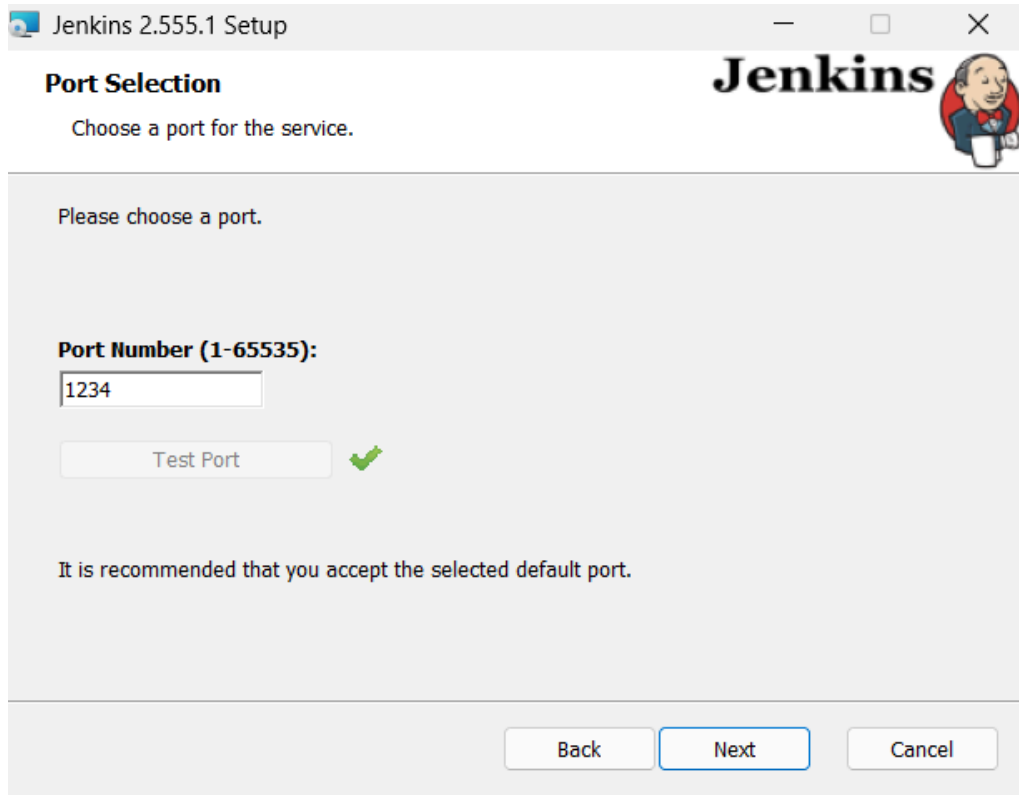
Account:

Password:

Test Credentials

Back Next Cancel

Port bebas, praktikum kali ini kita buat 1234 saja, klik test port untuk memastikan port tidak ada tabrakan. Dan pilih path java, bisa lihat di gambar. Next sampai Finish.



Jenkins 2.555.1 Setup

Port Selection

Choose a port for the service.

Please choose a port.

Port Number (1-65535):

Test Port 

It is recommended that you accept the selected default port.

Back Next Cancel



Jenkins 2.555.1 Setup

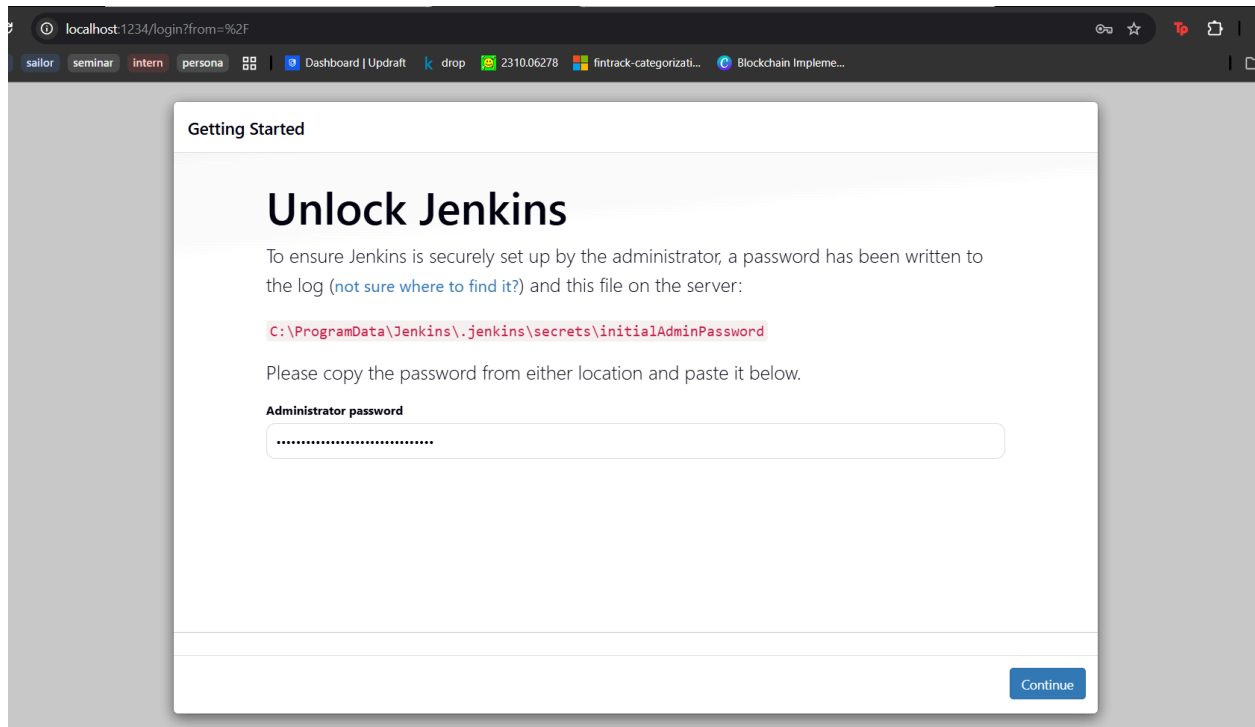
Select Java home directory (JDK or JRE)

Please select the path of a Java Development Kit or Java Runtime Environment. Only Java 21 and 25 are supported by Jenkins.

Change...

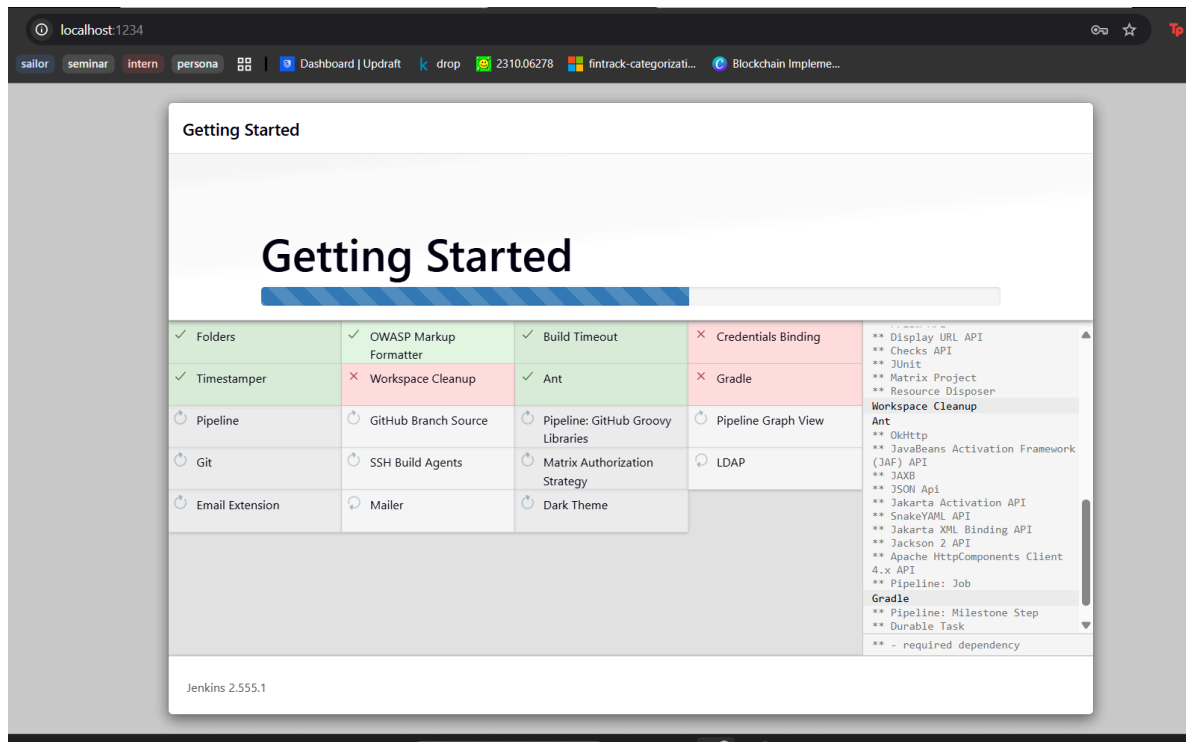
Back Next Cancel

Akses localhost:1234 untuk Web UI Jenkins



Perlu Unlock Jenkins untuk initial setup, cari administrator password sesuai perintah di layar kalian.

Kalau sudah unlock Jenkins, nanti Pilih "Install suggested plugins" dan akan memulai instalasi



Bikiin admin user, untuk praktikum kita bikin username: **admin**, password: **admin** saja, email isi pribadi saja.

Getting Started

Create First Admin User

Username

admin

Password

.....

Confirm password

Full name

E-mail address

Jenkins 2.555.1

Skip and continue as admin

Save and Continue

Next sampai muncul tampilan dashboard

 Jenkins

[+ New Item](#)

[Build History](#)

Build Queue

No builds in the queue.

Build Executor Status

0/2

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job

Set up a distributed build

Set up an agent

Configure a cloud

Learn more about distributed builds

[Add description](#)

REST API

Jenkins 2.555.1

Jika instalasi plugins ada yang gagal, bisa ikuti hal-hal berikut.

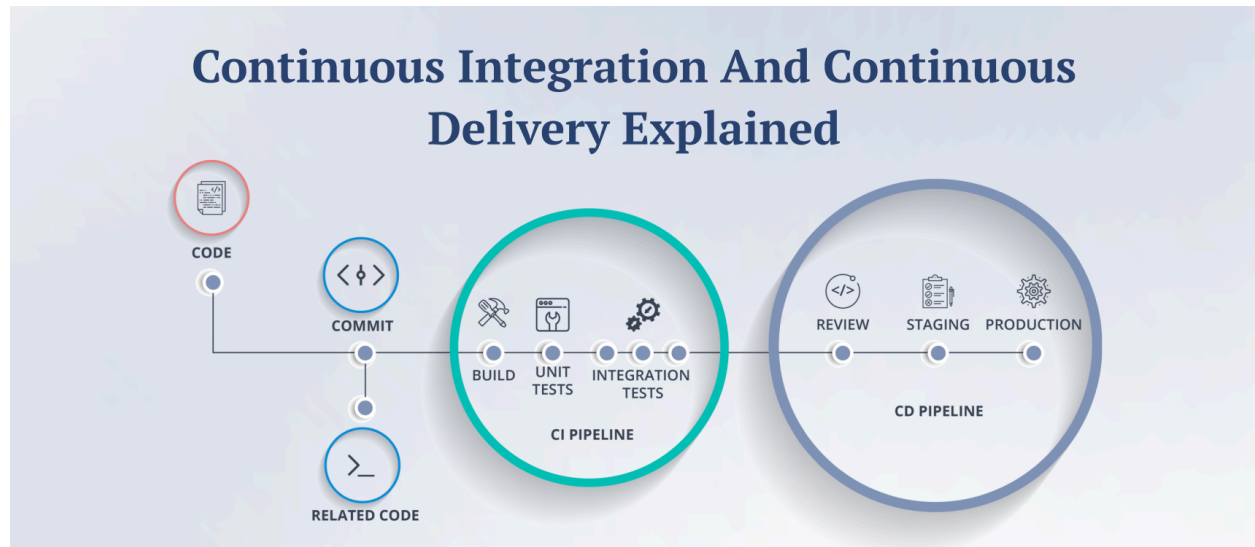
Download [1.1.1.1](#) atau coba pakai VPN. Uninstall Jenkins:

1. Stop Service: Buka *Task Manager* -> tab *Services*, cari Jenkins, klik kanan lalu Stop.
2. Uninstall: Pergi ke *Control Panel* -> *Programs and Features*, lalu Uninstall Jenkins.
3. Hapus Folder Data: Buka File Explorer dan hapus folder berikut (jika masih ada):
 - C:\Program Files\Jenkins
 - C:\ProgramData\Jenkins (Folder ini biasanya tersembunyi, langsung ketik saja di address bar).
 - C:\Users\[User]\.jenkins (Cek juga di sini, hapus jika ada).

Lalu install kembali Jenkins nya

CI/CD

Praktik DevOps adalah bagian otomatisasi dalam Software Development Life Cycle (SDLC) yang membagi proses menjadi dua tahap utama.



<https://www.encodedots.com/blog/what-is-ci-cd>

CI (Continuous Integration)

Praktik mengintegrasikan perubahan kode ke dalam repositori utama secara rutin. Proses ini mencakup Automated Build (membangun aplikasi) dan Automated Testing (menjalankan unit test) untuk memastikan kode baru tidak merusak sistem yang ada.

CD (Continuous Delivery/Deployment)

Tahap lanjutan di mana hasil *build* yang sudah lolos uji coba otomatis langsung di-deploy ke lingkungan produksi atau staging tanpa campur tangan manual.

Jenkins dalam Ekosistem CI/CD

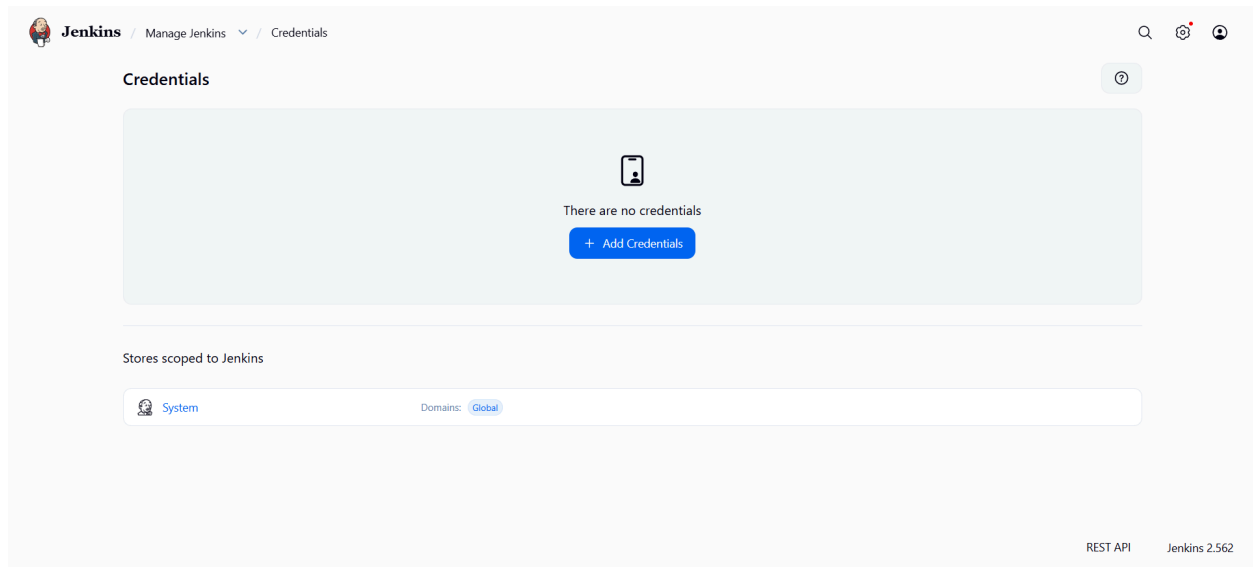
Sebuah automation server atau alat orkestrasi. Dalam alur CI/CD, Jenkins berperan sebagai koordinator yang menjalankan instruksi-instruksi berikut secara berurutan:

Tahapan	Peran Jenkins	Output
Trigger	Mendeteksi adanya <i>push</i> kode baru dari developer ke repositori (Git).	Sinyal mulai.

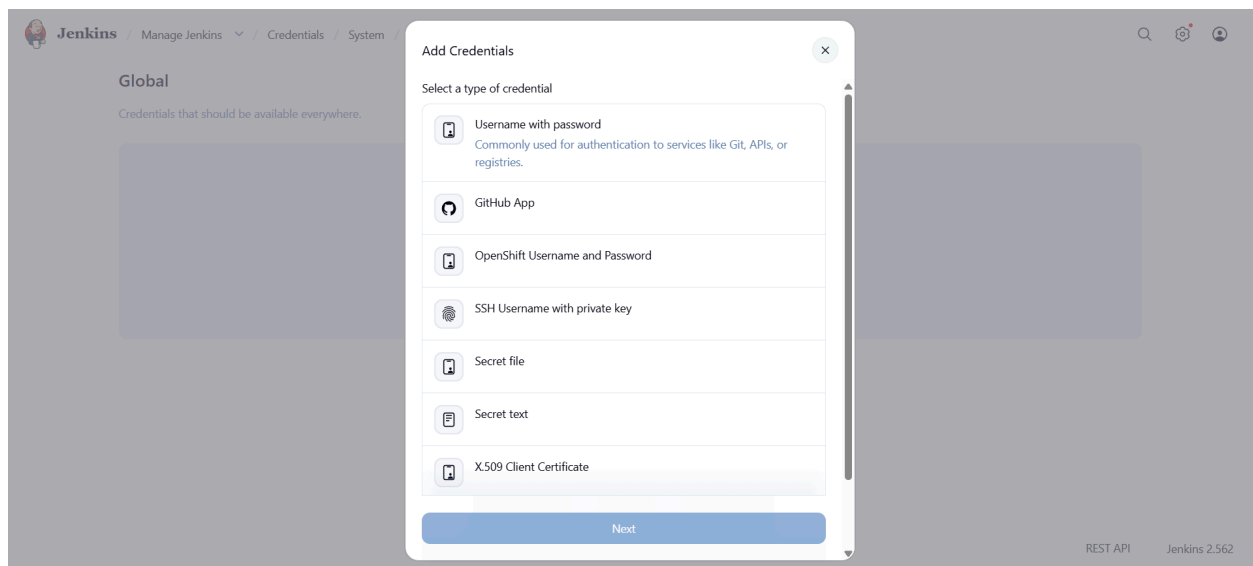
Build	Mengambil kode, lalu menjalankan perintah <i>build</i> (misal: <code>docker build</code>).	Container Image.
Test	Menjalankan pengujian otomatis (Unit Test/Security Scan).	Laporan kelulusan tes.
Push	Mengirimkan <i>image</i> yang sudah jadi ke Container Registry (seperti Docker Hub).	Image siap pakai.
Deploy	Memerintah Kubernetes (melalui <code>kubectl</code>) untuk memperbarui Pod dengan image terbaru.	Aplikasi ter-update.

IMPLEMENTASI 1 (CI/CD menggunakan Jenkins)

Persiapan kredensial (untuk push docker hub) dengan klik Manage Jenkins -> Credentials

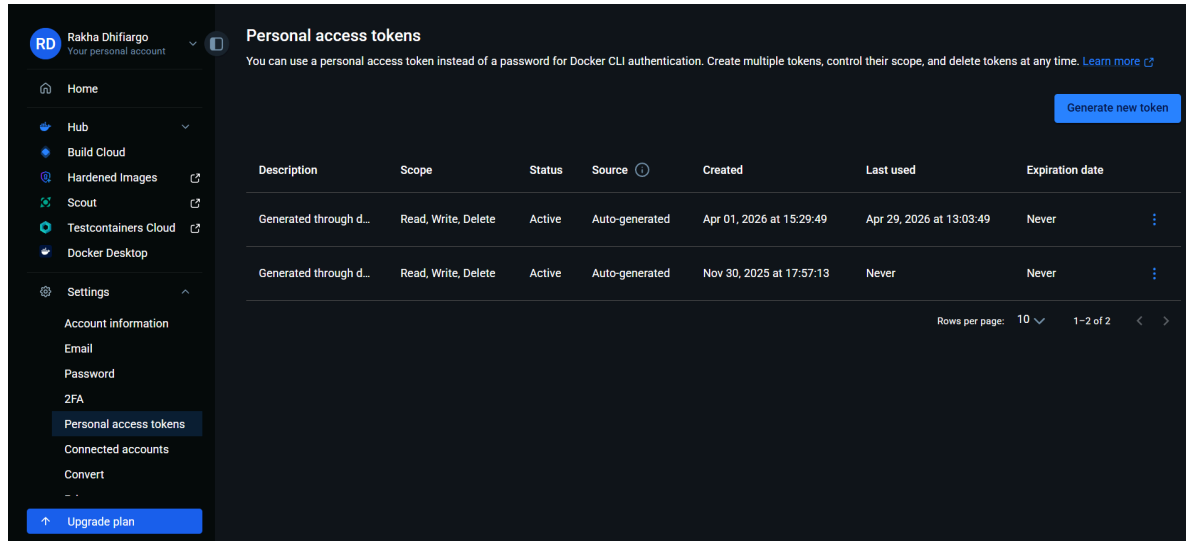


Klik Domains: [Global](#) -> Add Credential, dan nanti akan muncul pop up seperti ini

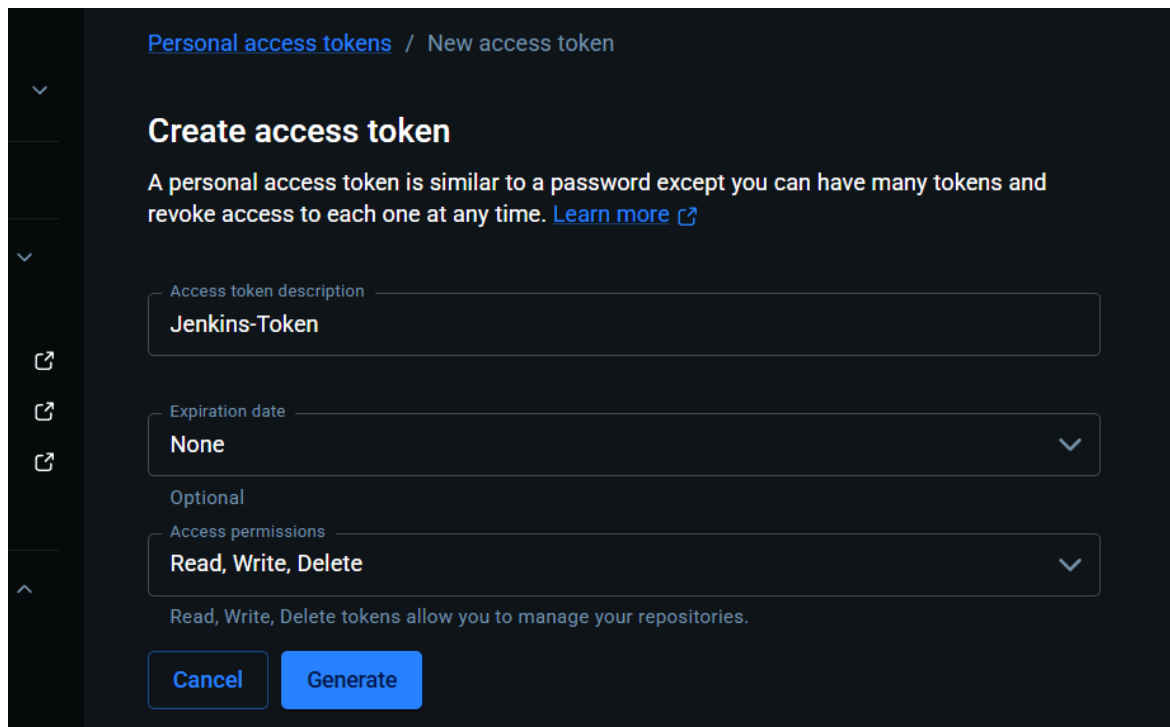


Sebelum lanjut, disini kita akan membuat Personal Access Token dari akun docker hub kita untuk keperluan kredensial di Jenkins.

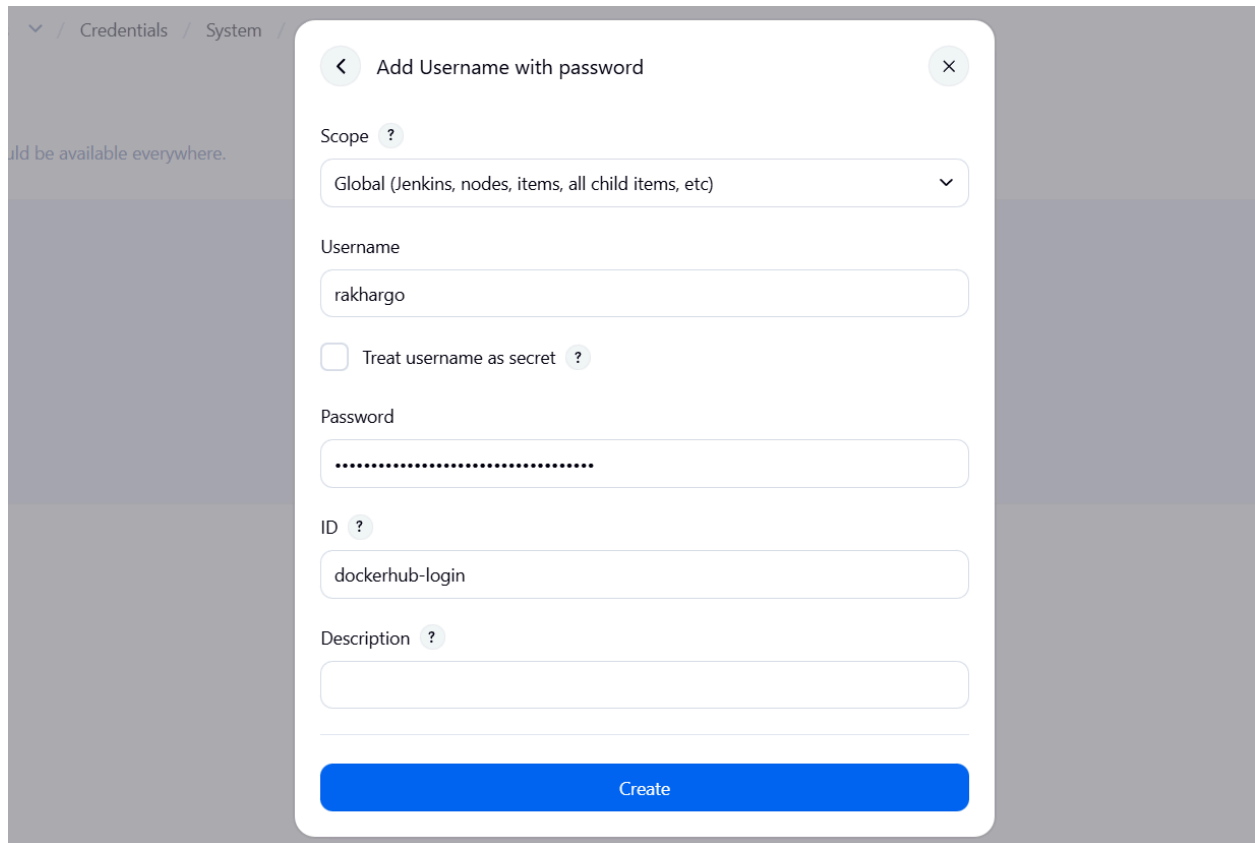
Pergi ke hub.docker.com lalu login, Profil (pojok kanan atas) -> Account Settings -> Personal access tokens dan muncul halaman berikut



Klik Generate new token dan isi nama, expiration date bikin none saja, yang penting access permissions bikin Read, Write, Delete. Klik Generate dan **simpan tokennya (dckr_pat_xxx...)** baik-baik, jangan sampai bocor, kalau hilang atau bocor bisa hapus yang lama lalu create baru. Token inilah yang akan menjadi alternatif password kita nanti.



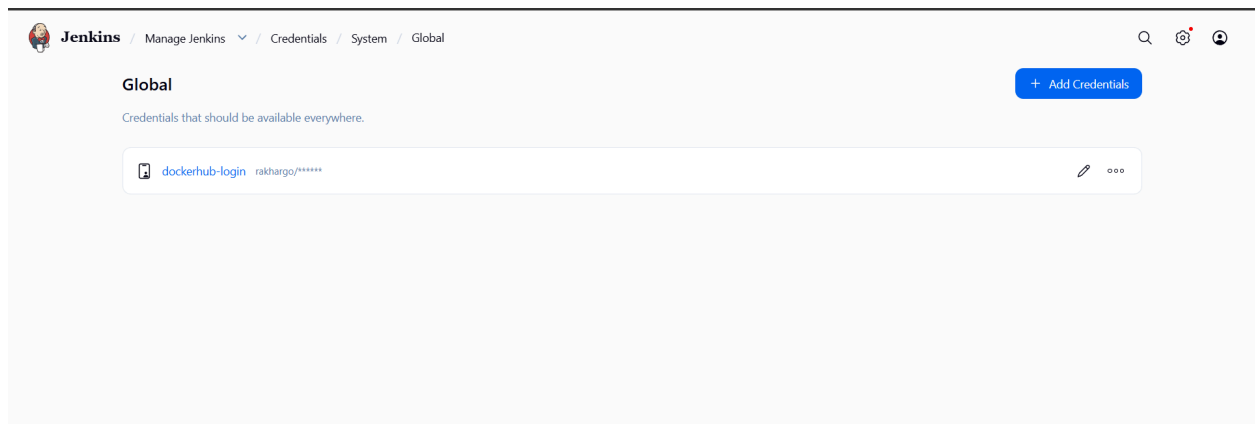
Isi username sama dengan docker hub, lalu password dengan PAT yang di generate tadi dan ID dockerhub-login (label dipanggil di script nanti)



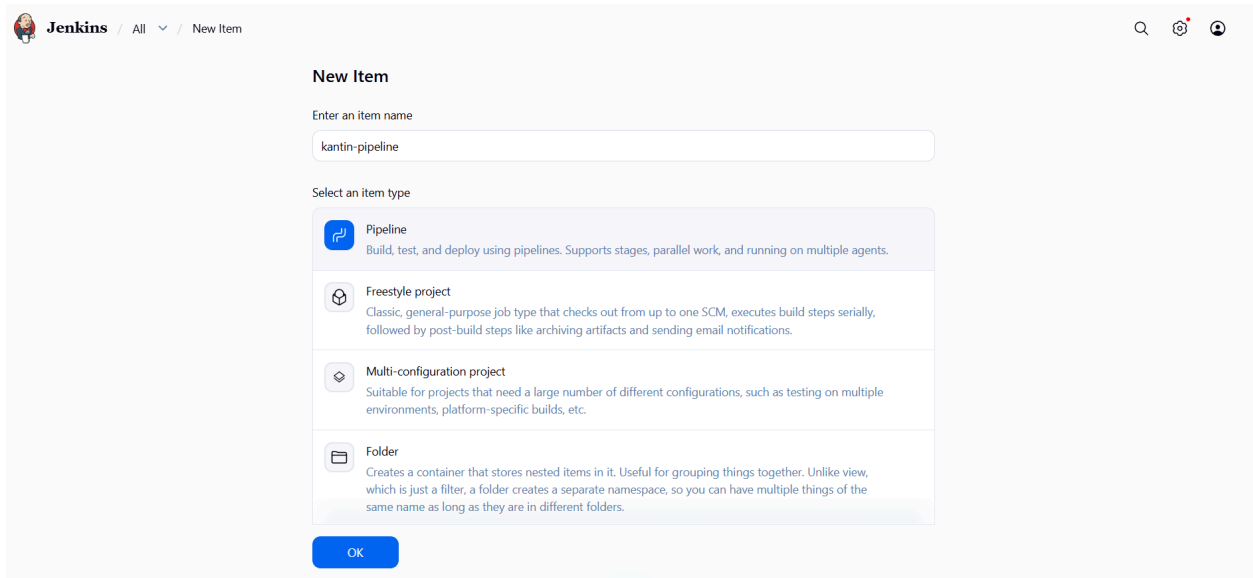
The screenshot shows a modal dialog box titled "Add Username with password" in Jenkins. The dialog has a back arrow and a close 'X' button. It contains the following fields and options:

- Scope**: A dropdown menu with a question mark icon, currently set to "Global (Jenkins, nodes, items, all child items, etc)".
- Username**: A text input field containing the value "rakhargo".
- Treat username as secret**: An unchecked checkbox with a question mark icon.
- Password**: A password input field with masked characters (dots).
- ID**: A text input field with a question mark icon, containing the value "dockerhub-login".
- Description**: A text input field with a question mark icon, currently empty.
- Create**: A large blue button at the bottom right of the dialog.

Hasil credential yang dibuat (Credentials Binding)



Ke dashboard, Klik New Item (kiri atas), masukkan item name kantin-pipeline lalu pilih item type “Pipeline”. Tipe ini digunakan untuk membuat alur kerja otomatis dari build sampai deploy menggunakan skrip. Klik OK.



New Item

Enter an item name

kantin-pipeline

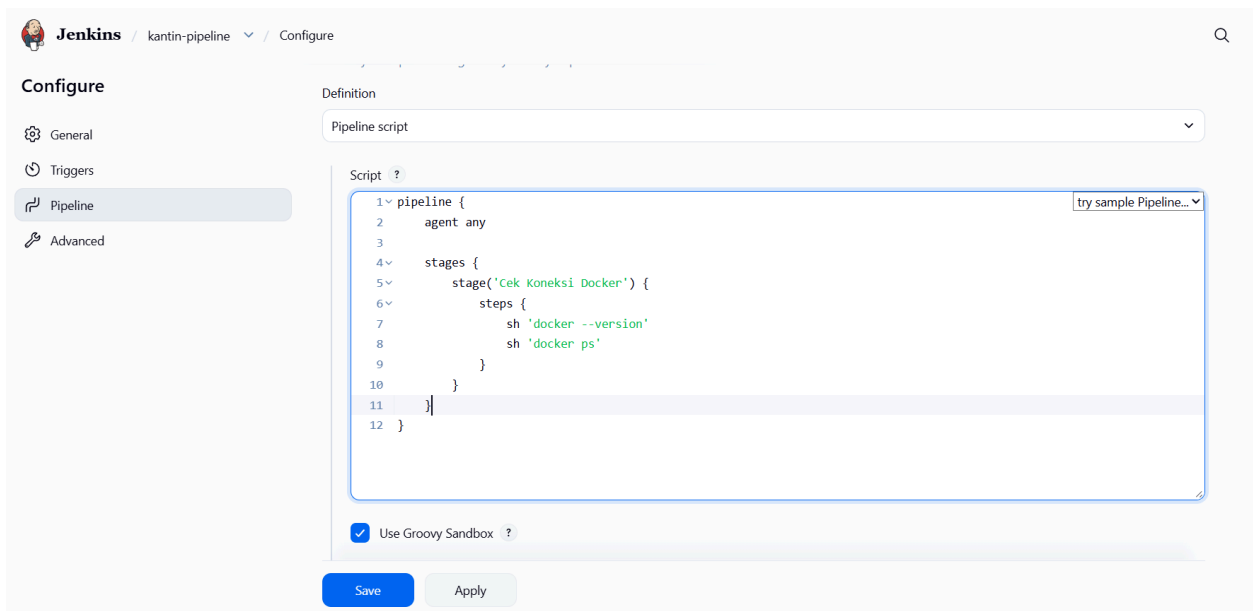
Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Proses ini adalah bagian dari Infrastructure as Code (IaC) dan Automated Build. Kita tidak ingin mengetik docker build secara manual selamanya, kita ingin Jenkins yang melakukannya setiap kali ada perubahan kode.

Coba jalankan pipeline dengan masuk ke kantin-pipeline -> Configure, disini kita akan sekaligus verifikasi docker yang di dalam Jenkins. Ke bagian Pipeline dan disini kita akan masukkan script sederhana untuk cek docker.



Configure

General
Triggers
Pipeline
Advanced

Definition

Pipeline script

Script ?

```
1 pipeline {  
2   agent any  
3  
4   stages {  
5     stage('cek koneksi Docker') {  
6       steps {  
7         sh 'docker --version'  
8         sh 'docker ps'  
9       }  
10    }  
11  }  
12 }
```

☒ Use Groovy Sandbox ?

Save Apply

Masukkan script berikut

```
pipeline {
  agent any

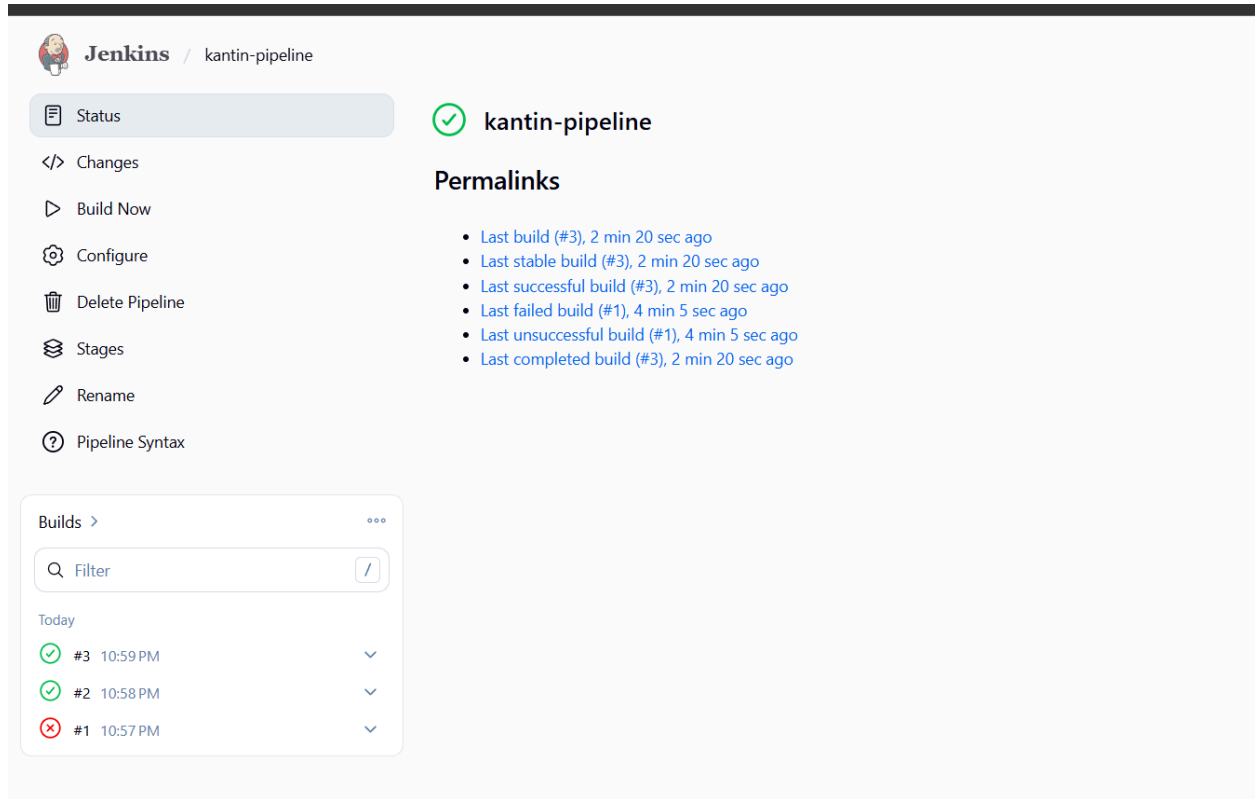
  stages {
    stage('Cek Koneksi Docker') {
      steps {
        // gunakan 'bat' untuk Windows Native, 'sh' untuk linux debian,
dan seterusnya
        bat 'docker --version'
        bat 'docker ps'
      }
    }
  }
}
```

pipeline { ... }: Ini adalah blok utama untuk mendefinisikan alur kerja otomatis nya.

agent any: Memberitahu Jenkins untuk menjalankan tugas ini di mesin mana pun yang tersedia (dalam hal ini, di dalam kontainer Jenkins itu sendiri).

stages { ... }: Blok yang mendefinisikan tahapan pengerjaan (seperti Build, Test, Deploy) untuk memvisualisasikan dan mengelola alur. Berisi satu atau lebih stage, yang masing-masing berisi steps (langkah spesifik/perintah) yang dieksekusi secara sekuensial.

Setelah masukkan script, kita bisa klik “Save”. Setelah itu, di menu sebelah kiri klik “Build now”, nanti akan ada history Builds yang di bawah, klik yang terbaru (yang script docker)



Jenkins / kantin-pipeline

Status

Changes

Build Now

Configure

Delete Pipeline

Stages

Rename

Pipeline Syntax

kantin-pipeline

Permalinks

- Last build (#3), 2 min 20 sec ago
- Last stable build (#3), 2 min 20 sec ago
- Last successful build (#3), 2 min 20 sec ago
- Last failed build (#1), 4 min 5 sec ago
- Last unsuccessful build (#1), 4 min 5 sec ago
- Last completed build (#3), 2 min 20 sec ago

Builds >

Filter

Today

- #3 10:59 PM
- #2 10:58 PM
- #1 10:57 PM

Buka Console Output untuk lihat hasilnya, jika sudah maka sudah berhasil sambung ke docker



Jenkins / kantin-pipeline / #3

Status

Changes

Console Output

Edit Build Information

Delete build '#3'

Timings

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Console

Download Copy View as plain text

```
Started by user admin local
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\jenkins\workspace\kantin-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (cek Koneksi Docker)
[Pipeline] bat

C:\ProgramData\Jenkins\jenkins\workspace\kantin-pipeline>docker --version
Docker version 29.3.1, build c2be9cc
[Pipeline] bat

C:\ProgramData\Jenkins\jenkins\workspace\kantin-pipeline>docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jika sudah, kita lanjut ke app yang kita bangun sebelumnya yaitu kantin-app. Kita akan memanfaatkan platform cloud yaitu Github disini untuk menjadi repositori dari kode kita. Perlu menambah dan mengubah beberapa hal.

frontend/index.html

```
const API_URL = '/api';
```

kantin-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kantin-ingress
  # annotations:
  #   nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: kantin.local
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: backend-service
            port:
              number: 5000
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
```

kantin-k8s.yaml (ubah sesuai username dockerhub)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-kantin
spec:
```

```
replicas: 3
selector:
  matchLabels:
    app: backend
template:
  metadata:
    labels:
      app: backend
  spec:
    containers:
      - name: backend-api
        image: rakhargo/kantin-backend:latest
        imagePullPolicy: Always
        ports:
          - containerPort: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-kantin
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
```



```

metadata:
  labels:
    app: frontend
spec:
  containers:
  - name: frontend-ui
    image: rakhargo/kantin-frontend:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
  type: ClusterIP

```

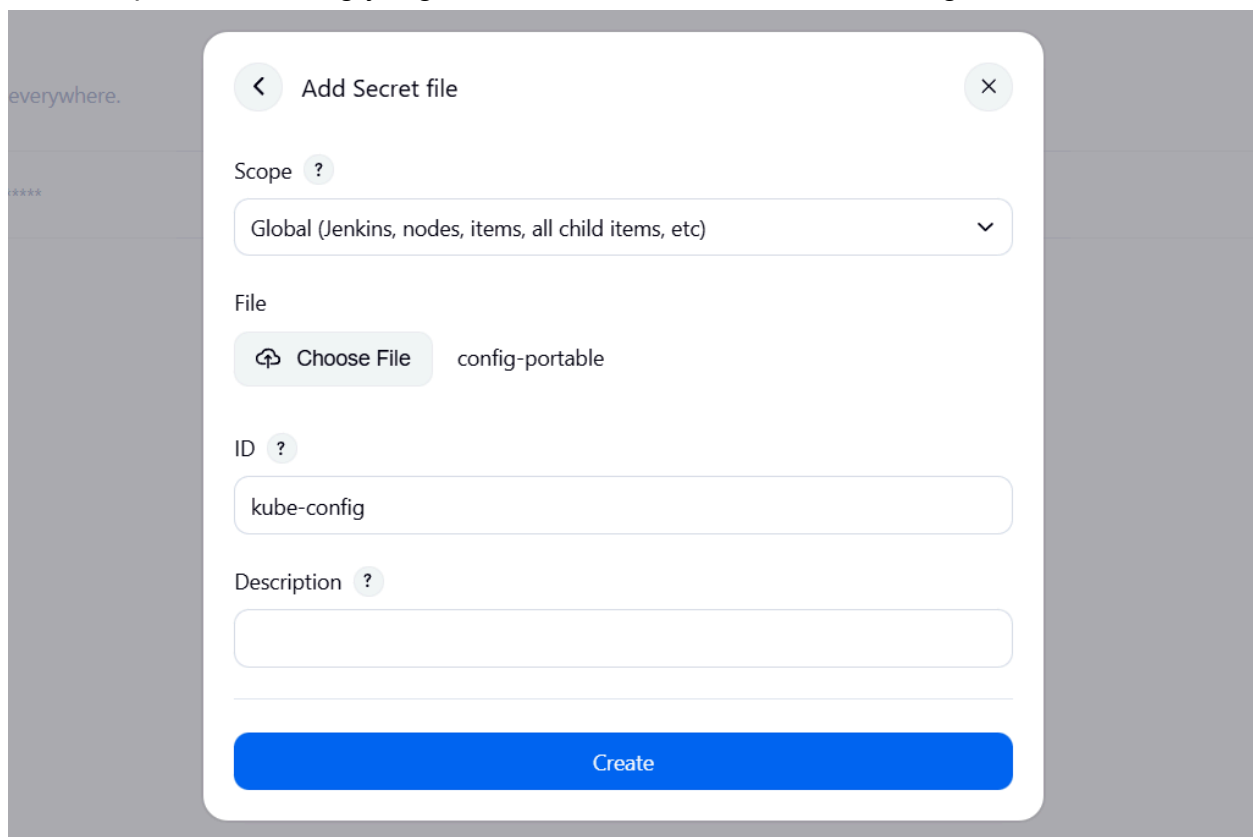
Push kode kantin-app yang sebelumnya ke repository public Github.

The screenshot shows the GitHub repository page for 'rakhargo/kantin-app'. The repository is public and has 0 stars, 0 forks, and 0 watchers. The main branch is 'main'. The repository contains several files and folders, including 'backend', 'frontend', 'config-portable', 'kanting-ingress.yaml', 'kanting-k8s.yaml', and 'tes.groovy'. The 'README' file is highlighted. The repository was updated 3 minutes ago by 'rakhargo' with 4 commits. The page also shows the 'About' section with no description, website, or topics provided, and the 'Releases' section with no releases published.

Sekarang kita akan mengurus config kube nya, karena tanpa file kubeconfig, perintah kubectl akan otomatis mencari server di `http://localhost:8080`. Karena Jenkins kita juga jalan di port 8080, terjadilah tabrakan.

1. Jalankan `minikube start`
2. Pergi ke suatu direktori sementara, misalnya workspace kantin-app kita
3. Jalankan command berikut `kubectl config view --flatten --minify > config-portable` (Setiap restart minikube akan selalu berubah terutama portnya jadi kalau bisa minikube cukup sekali start)

Lanjut kita masukkan Config ke Jenkins via Pipeline sebagai Secret File. Ke Dashboard Jenkins -> Manage Jenkins -> Credentials. Klik (global) -> Add Credentials. Pilih Secret file dan upload file config yang baru diubah tadi, beri ID kube-config.



The screenshot shows the 'Add Secret file' dialog in Jenkins. The dialog has a title bar with a back arrow and a close button. Below the title bar, there is a 'Scope' dropdown menu set to 'Global (Jenkins, nodes, items, all child items, etc)'. Below that is a 'File' section with a 'Choose File' button and the text 'config-portable'. Below the file section is an 'ID' field with the text 'kube-config'. Below the ID field is a 'Description' field. At the bottom of the dialog is a blue 'Create' button.

Menulis Jenkinsfile. Sekarang, balik ke dashboard Jenkins, masuk ke proyek kantin-pipeline, klik Configure, lalu ganti skrip tes docker tadi dengan Jenkinsfile berikut

```
pipeline {
  agent any

  environment {
    // Ganti dengan username Docker Hub yang sesuai
    DOCKER_HUB_USER = 'rakhargo'
```

```

// ID kredensial yang dibuat di 'Add Credentials' tadi
DOCKER_HUB_ID = 'dockerhub-login'
// URL repo GitHub aplikasi
GIT_REPO_URL = 'https://github.com/rakhargo/kantin-app.git'
}

stages {
    stage('Checkout Code') {
        steps {
            // Tahap mengambil kode terbaru dari repositori, pastikan branch
sesuai yang digunakan
            git branch: 'main', url: "${GIT_REPO_URL}"
        }
    }

    stage('Build Images') {
        steps {
            withCredentials([usernamePassword(credentialsId:
"${DOCKER_HUB_ID}",
                passwordVariable: 'PASS',
                usernameVariable: 'USER')]) {
                bat 'echo %PASS%| docker login -u %USER% --password-stdin'

                bat "docker build -t %DOCKER_HUB_USER%/kantin-backend:latest
./backend"

                bat "docker build -t %DOCKER_HUB_USER%/kantin-frontend:latest
./frontend"
            }
        }
    }

    stage('Push to Docker Hub') {
        steps {
            withCredentials([usernamePassword(credentialsId:
"${DOCKER_HUB_ID}",
                passwordVariable: 'PASS',
                usernameVariable: 'USER')]) {
                bat "echo %PASS%| docker login -u %USER% --password-stdin"
                bat "docker push %DOCKER_HUB_USER%/kantin-backend:latest"
            }
        }
    }
}

```

```

        bat "docker push %DOCKER_HUB_USER%/kantin-frontend:latest"
    }
}

stage('Deploy to Kubernetes') {
    steps {
        echo 'Deploying to Kubernetes using Secret File...'
        // Jenkins akan mengambil file 'kube-config' dan menaruhnya di
        folder temporary
        withCredentials([file(credentialsId: 'kube-config', variable:
        'KUBECONFIG_FILE')]) {
            // Kita panggil kubectl dengan bendera --kubeconfig yang
            mengarah ke file temp tadi
            bat 'kubectl --kubeconfig=%KUBECONFIG_FILE% apply -f
            kantin-k8s.yaml --validate=false'
            bat 'kubectl --kubeconfig=%KUBECONFIG_FILE% apply -f
            kantin-ingress.yaml --validate=false'
        }
    }
}
}
}

```

Sebelum di build, ada beberapa hal yang harus dilakukan terkait networking, disini kita akan menerapkan Ingress (semacam API Gateway). Buka terminal as administrator, jalankan `minikube addons enable ingress` dan dilanjut `minikube tunnel`, jangan dimatikan terminalnya.

```

D:\codinglucu\Asprak\cloud\2\kantin-app>minikube addons enable ingress
🔔 ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
🔔 After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
  ▪ Using image registry.k8s.io/ingress-nginx/controller:v1.14.3
  ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
  ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
🔵 Verifying ingress addon...
🌟 The 'ingress' addon is enabled

D:\codinglucu\Asprak\cloud\2\kantin-app>kubectl get pods -n ingress-nginx
NAME                                READY   STATUS    RESTARTS   AGE
ingress-nginx-admission-create-nnjxx 0/1     Completed 0           31s
ingress-nginx-admission-patch-rjvfr   0/1     Completed 0           31s
ingress-nginx-controller-596f8778bc-f776h 0/1     Running   0           31s

```

Ingress itu anggap saja sebagai "Resepsionis". Setiap aplikasi punya nomor port yang beragam, semua orang cukup datang ke satu alamat (kantin.local), dan Ingress yang akan mengarahkan ke ruangan (Service) yang benar.

Daftarkan Domain di Hosts File Windows

Karena kantin.local bukan domain asli di internet, kita harus "menipu" Windows agar tahu kalau domain itu ada di IP Minikube.

1. Buka Notepad sebagai Administrator.
2. Buka file: `C:\Windows\System32\drivers\etc\hosts`.
3. Tambahkan baris ini di paling bawah: `127.0.0.1 kantin.local`

Dan akhirnya baru kita "Build Now" kantin-pipeline yang di Jenkins

```
[Pipeline] echo
Deploying to Kubernetes using Secret File...
[Pipeline] withCredentials
Masking supported pattern matches of %KUBECONFIG_FILE%
[Pipeline] {
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\kantIn-pipeline>kubectl --kubeconfig=**** apply -f kantin-k8s.yaml --validate=false
deployment.apps/backend-kantin unchanged
service/backend-service unchanged
deployment.apps/frontend-kantin unchanged
service/frontend-service unchanged
[Pipeline] bat

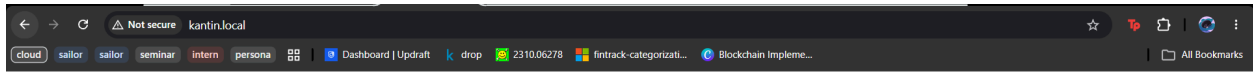
C:\ProgramData\Jenkins\.jenkins\workspace\kantIn-pipeline>kubectl --kubeconfig=**** apply -f kantin-ingress.yaml --validate=false
ingress.networking.k8s.io/kantin-ingress created
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Sedang direalisasikan

```
D:\codinglucu\Asprak\cloud\2\kantIn-app>kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
backend-kantin-65848dd795-7959x     1/1     Running             0          24s
backend-kantin-65848dd795-g5xp2     0/1     ContainerCreating   0          3s
backend-kantin-65848dd795-sb9t8     1/1     Running             0          12s
backend-kantin-7d75ddcc57-4sbvz     0/1     ImagePullBackOff    0          2m48s
frontend-kantin-d4bc6fb74-9x7hq     1/1     Running             0          24s

D:\codinglucu\Asprak\cloud\2\kantIn-app>
```

Buka kantin.local



Kantin FPMIPA

- Nasi Goreng
- Es Teh
- Gorengan

Tambah Menu Baru:

Dan ini hasilnya

Karakteristik Cloud Native

Pada tahap ini, kita tidak lagi hanya menjalankan kontainer, tapi memastikan aplikasi memiliki sifat:

- Distributed: Berjalan di berbagai node yang saling bekerja sama.
- Resilient & Scalable: Tahan banting terhadap kegagalan dan mudah ditambah kapasitasnya.
- Observable: Mudah dipantau melalui log dan metrik.
- Self-Healing: Secara otomatis memperbaiki diri jika terjadi crash.

Konsep dan Terms baru:

1. Health Checks (Probes): Sensor untuk memantau kesehatan aplikasi di dalam Pod.
2. Resource Management: Pengaturan jatah CPU dan RAM agar cluster tetap stabil.
3. Ingress: Gerbang masuk utama untuk mengatur lalu lintas URL (Routing).
4. Rolling Update: Strategi memperbarui aplikasi tanpa mematikan layanan (*zero downtime*).

IMPLEMENTASI 2 (Contoh Advanced Container Orchestration)

Note: Implementasi 2 hanya contoh kalau ada terms dan fitur-fitur baru, belum untuk diimplementasikan

Minikube dari modul sebelumnya

```
minikube start --driver=docker
```

```
minikube addons enable ingress
```

kantin-k8s.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-kantin
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
```

```

    app: backend
spec:
  containers:
  - name: backend-api
    image: kantin-backend:latest
    # --- TAMBAHAN MODUL 2 ---
    resources:
      limits:
        memory: "256Mi" # Batas maksimal RAM
        cpu: "500m"      # Batas maksimal CPU (0.5 core)
      livenessProbe:      # Mengecek apakah aplikasi hang
        httpGet:
          path: /api/info
          port: 5000
        initialDelaySeconds: 10
        periodSeconds: 15
        readinessProbe:    # Mengecek apakah aplikasi siap terima
traffic
        httpGet:
          path: /api/info
          port: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
  - protocol: TCP
    port: 5000
    targetPort: 5000
  type: LoadBalancer

```



```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-kantin
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend-ui
          image: kantin-frontend:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: LoadBalancer
```

kantin-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kantin-ingress
spec:
  rules:
  - http:
      paths:
      - path: /api # Semua traffic ke /api masuk ke Backend
        pathType: Prefix
        backend:
          service:
            name: backend-service
            port:
              number: 5000
      - path: / # Traffic utama masuk ke Frontend
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
```

```
docker build -t kantin-backend:latest ./backend
docker build -t kantin-frontend:latest ./frontend
```

```
kubectl apply -f kantin-k8s.yaml
kubectl apply -f kantin-ingress.yaml
```

```
kubectl get pods (cek deploy)
```

Buka terminal baru (bebas) jalankan `minikube tunnel`

```

D:\codinglucu\Asprak\cloud>minikube tunnel
[✓] Tunnel successfully started

✖ NOTE: Please do not close this terminal as this process must stay alive for the tunnel to be accessible ...

🔗 Starting tunnel for service backend-service.
! Access to ports below 1024 may fail on Windows with OpenSSH clients older than v8.1. For more information, see: https://minikube.sigs.k8s.io/docs/handbo
ok/accessing/#access-to-ports-1024-on-windows-requires-root-permission
🔗 Starting tunnel for service frontend-service.
! Access to ports below 1024 may fail on Windows with OpenSSH clients older than v8.1. For more information, see: https://minikube.sigs.k8s.io/docs/handbo
ok/accessing/#access-to-ports-1024-on-windows-requires-root-permission
🔗 Starting tunnel for service kantin-ingress.

```

kubectl get ingress

```

PS D:\codinglucu\Asprak\cloud\3\kantın-app> kubectl get ingress
NAME          CLASS    HOSTS      ADDRESS          PORTS    AGE
kantın-ingress  nginx    *          192.168.49.2     80       3m33s

```

kubectl describe pod <nama-pod-backend>

Implementasi AKS

code

<https://github.com/ssamu11/praktikum-kantin-app.git>

TAHAP 1: Membuat Server Kubernetes via Azure Portal

1. Login ke portal.azure.com.
2. Cari dan pilih Kubernetes services di bilah pencarian.
3. Klik + Create → Create a Kubernetes cluster.
4. Di tab Basics, atur:
 - Resource group: Buat baru, beri nama Praktikum-AKS-RG.
 - Cluster preset: Pilih Dev/Test.
 - Cluster name: Ketik kantin-cluster.
 - Region: Pilih Southeast Asia.
5. Di bagian Primary node pool, klik Change size. Pilih Standard_D2s_v3 (Wajib agar server tidak kehabisan CPU).
6. Klik Review + create → klik Create. Tunggu sekitar 5-10 menit.
7. Buka terminal laptop, sambungkan ke server AKS:

```
az aks get-credentials --resource-group Praktikum-AKS-RG --name kantin-cluster
```

TAHAP 2: Deployment Manual Awal (Mendapatkan IP Publik)

Kita harus memancing Azure untuk memberikan IP Publik kepada Backend kita agar Frontend tahu harus mengambil data dari mana.

1. Buka folder kantin-app di VSCode.
2. Buka file kantin-k8s.yaml dan sesuaikan:
 - Hapus bagian resources: (limits memory & cpu) pada *backend*.
 - Tambahkan imagePullPolicy: Always pada kontainer *backend* dan *frontend*.
 - Ubah replicas: 1 pada *backend*.
3. Buka terminal VSCode, jalankan file YAML tersebut secara manual:

```
kubectl apply -f kantin-k8s.yaml
```

4. Cek dan tunggu sampai IP Backend muncul:

```
kubectl get svc backend-service
```

5. Catat EXTERNAL-IP dari backend-service (Contoh: 135.171.161.212).

TAHAP 3: Menyambungkan Frontend & Push ke GitHub

1. Buka file frontend/index.html (atau file *script* JS kamu) di VSCode.
2. Ubah variabel URL API menggunakan IP yang baru saja kamu dapatkan:

```
const API_URL = 'url pod backend kalian';
```

3. Buka repositori kosong di GitHub bernama praktikum-kantin-app (Public, tanpa README).
4. Di terminal VSCode, inialisasi Git dan lempar kode ke github

```
git init
git add .
git commit -m "Inisialisasi kode dengan IP Backend Azure"
git branch -M main
git remote add origin <URL_GITHUB_KAMU>
git push -u origin main
```

5. Buat Token Docker Hub: Login ke hub.docker.com → Account Settings → Personal access tokens. Buat token baru (Jenkins-Token) dengan izin Read, Write, Delete. Simpan tokennya.

TAHAP 4: Menyiapkan Jenkins & Ngrok

1. Buka terminal, jalankan Jenkins:

```
docker run -d -p 1234:8080 -p 50000:50000 --name jenkins-server
--restart=on-failure -u root -v
/var/run/docker.sock:/var/run/docker.sock jenkins/jenkins:latest
```

Note: Port 1234 terpakai di Jenkins Native yang kita implementasi lokal tadi. Bisa diubah aja dengan yang kosong misal 1235, dll.

2. Install (Docker & Kubectl) di dalam Jenkins:

```
docker exec -it jenkins-server bash
apt-get update && apt-get install -y docker.io
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
mv kubectl /usr/local/bin/
exit
```

3. Cek password awal Jenkins:

```
docker logs jenkins-server
```

4. Buka terminal baru, jalankan Ngrok untuk membuka tunnel Webhook:

[Download ngrok for Windows | ngrok](#)

```
ngrok http 1234
```

5. *Copy URL publik Ngrok (contoh: https://abcd.ngrok-free.app). Jangan tutup terminal ini!*

TAHAP 5: Konfigurasi Webhook & Web Jenkins

1. Buka repo GitHub kamu → Settings → Webhooks → Add webhook.
 - Payload URL: Paste link Ngrok dan tambah /github-webhook/. (Contoh: https://abcd.ngrok-free.app/github-webhook/)
 - Content type: application/json. Klik Add webhook.

2. Buka browser ke localhost:1234, masukkan password Jenkins, klik Install suggested plugins. Buat akun Admin.
3. Ke Manage Jenkins → Plugins → Available Plugins. Instal Docker Pipeline & Kubernetes CLI.
4. Ke Manage Jenkins → Credentials → System → Global credentials → Add Credentials:
 - Kunci Docker: Tipe *Username with password*. Isi Username Docker, isi password dengan Token Docker Hub, ID: dockerhub-login.
 - Kunci Azure AKS: Tipe *Secret file*. Upload file dari C:\Users\NAMA_USER\.kube\config, ID: aks-config.

TAHAP 6: Membuat CI/CD Pipeline

1. Di Dashboard Jenkins, klik New Item → Pipeline → Nama: Kantin-Deploy → OK.
2. Di Build Triggers, centang GitHub hook trigger for GITScm polling.
3. Di bagian Pipeline, masukkan skrip berikut (*Ganti Username Docker & URL GitHub!*):

```
Groovy
pipeline {
    agent any
    environment {
        DOCKER_USER = "GANTI_USERNAME_DOCKER_HUB"
        GIT_REPO_URL = "GANTI_URL_GITHUB_KAMU.git"
    }
    stages {
        stage('Checkout Code') {
            steps {
                git branch: 'main', url: "${GIT_REPO_URL}"
            }
        }
        stage('Build & Push Docker Image') {
            steps {
                withCredentials([usernamePassword(credentialsId:
                    'dockerhub-login', passwordVariable: 'PASS', usernameVariable:
                    'USER')]) {
                    sh "docker build -t ${USER}/kantin-backend:latest
                    ./backend"
                    sh "docker build -t
                    ${USER}/kantin-frontend:latest ./frontend"
```

```

        sh "echo ${PASS} | docker login -u ${USER}
--password-stdin"
        sh "docker push ${USER}/kantin-backend:latest"
        sh "docker push ${USER}/kantin-frontend:latest"
    }
}
}
stage('Deploy ke Azure AKS') {
    steps {
        withKubeConfig([credentialsId: 'aks-config']) {
            sh "kubectl apply -f kantin-k8s.yaml"
            sh "kubectl rollout restart deployment
backend-kantin"
            sh "kubectl rollout restart deployment
frontend-kantin"
        }
    }
}
}
}
}

```

4. Klik Save, lalu klik Build Now satu kali secara manual untuk memastikan Jenkins bisa berjalan sukses.

Test webhook

Buka terminal laptop, cek IP Frontend:

```
kubectl get svc frontend-service
```

```

git add .
git commit -m "Update nama kantin via Webhook"
git push origin main

```


Cleanup

```
az group delete --name Praktikum-AKS-RG --yes --no-wait
```